

Architecture Microservices

Tous les UC du cahier de charges · RabbitMQ · Spring Boot · Docker · API Gateway · Plan 4 semaines

Version complète — 29 May 2026

MICROSERVICES	UC COUVERTS	MEMBRES	MESSAGE BROKER	DÉPLOIEMENT
10 services + Gateway + Eureka	UC1 → UC42 (42 cas d'usage)	4 membres (bases Java)	RabbitMQ (AMQP)	Railway / Render

■■ Avertissement critique — à lire impérativement

42 cas d'usage, 10 microservices, RabbitMQ, API Gateway, Docker — avec 4 développeurs novices en 1 mois. Ce document ne minimise pas la difficulté : c'est objectivement une charge de travail pour une équipe senior de 8 personnes sur 3 mois. Le plan ci-dessous est le plus optimisé possible dans ces contraintes, avec une priorisation stricte P1/P2/P3 et un regroupement intelligent des services. Sans discipline absolue sur les priorités, le projet ne sera pas livrable.

1. Vue d'ensemble — Mapping UC → Microservices

Les 42 cas d'usage du cahier de charges sont distribués en 10 microservices métier + 2 services d'infrastructure (Gateway, Eureka). Chaque microservice a une responsabilité unique et des limites claires.

Service	Port	UC couverts	Responsabilité	Priorité
auth-service	8081	UC1, UC29, UC35	Inscription, connexion, OTP SMS, JWT, refresh token	■ P1
user-service	8082	UC17-18, UC28* UC30, UC31	Profils client/prestataire, validation admin, gestion users	■ P1
demande-service	8083	UC2-4, UC6-8 UC19-21, UC22-23	Création demandes, devis prestataire, validation client	■ P1
mission-service	8084	UC9-10, UC24, UC25	Cycle de vie mission, suivi temps réel (SSE)	■ P1
notation-service	8085	UC11, UC26	Notation bidirectionnelle, calcul moyenne	■ P2
litige-service	8086	UC12, UC34 UC36-39	Signalement litiges, traitement SC, résolution admin	■ P2
urgence-service	8087	UC13-16	Mode urgence, matching géolocalisé, contact direct	■ P2
paiement-service	8088	UC27, UC32 UC40-42	Mobile Money MTN/Orange, commissions, historique gains	■ P2
chat-service	8089	UC5	Messagerie temps réel (WebSocket STOMP)	■ P3
notification-service	8090	Transversal (tous UC)	Consumer RabbitMQ exclusif — SMS, push, email	■ P2
api-gateway	8080	Transversal	Routing, auth filter JWT, rate limiting, CORS	■ P1
eureka-server	8761	Infrastructure	Service discovery, health check, load balancing	■ P1

* UC28 Dashboard Prestataire = agrégation via Feign Client depuis user-service + demande-service + mission-service + paiement-service.

2. Architecture globale — Schéma complet

[illegible]

3. Détail complet des 10 microservices

Chaque fiche couvre : endpoints REST exacts, messages RabbitMQ produits/consommés, schéma de base de données, et responsable dans l'équipe.

:8081 auth-service UC: UC1 · UC29 · UC35	
Endpoints REST	POST /auth/register · POST /auth/login · POST /auth/logout POST /auth/verify-otp · POST /auth/resend-otp POST /auth/refresh-token · GET /auth/me
RabbitMQ PRODUIT	Routing key: user.registered → Exchange serviloc.events
RabbitMQ CONSOMME	Aucun (service source, ne consomme pas)
Tables BDD	users(id, email, tel, password_hash, role, otp_code, otp_expiry, is_verified, is_active, created_at, updated_at)
Responsable	Membre 1 — livraison OBLIGATOIRE fin Semaine 1

:8082 user-service UC: UC17-18 · UC28* · UC30 · UC31	
Endpoints REST	GET/PUT /users/{id} · GET /users (admin) POST/GET/PUT /prestataires/{id} · GET /prestataires?zone=&metier;= PUT /prestataires/{id}/valider · PUT /prestataires/{id}/rejeter GET /prestataires/{id}/documents · POST /prestataires/{id}/documents GET /clients/{id} · DELETE /users/{id} (admin)
RabbitMQ PRODUIT	Routing key: prestataire.validated, prestataire.rejected
RabbitMQ CONSOMME	Routing key: user.registered → crée le profil initial vide
Tables BDD	user_profiles(user_id, nom, prenom, avatar, zone, created_at) prestataire_profiles(user_id, metier, description, zone, tarif, statut_validation, note_moyenne, nb_missions) documents(id, presta_id, type_doc, url_fichier, statut)
Responsable	Membre 2

:8083 demande-service UC: UC2-3-4 · UC6-7-8 · UC19-21 · UC22-23	
Endpoints REST	POST /demandes · GET /demandes · GET /demandes/{id} PUT /demandes/{id}/annuler · GET /demandes?zone=&metier;=&statut;= (presta) POST /demandes/{id}/devis · GET /demandes/{id}/devis GET /devis/{id} · PUT /devis/{id}/valider · PUT /devis/{id}/refuser PUT /devis/{id}/modifier
RabbitMQ PRODUIT	Routing keys: demande.created, devis.created, devis.validated, devis.refused
RabbitMQ CONSOMME	Routing key: devis.validated → déclenche création mission dans mission-service (via queue q.mission.from.devis liée à mission-service)
Tables BDD	demandes(id, client_id, metier, description, zone, statut, urgence, budget_max, date_souhaitee, created_at) devis(id, demande_id, presta_id, montant, details, statut, delai_jours, created_at)
Responsable	Membre 2 (semaines 2-3)

:8084 mission-service UC: UC9-10 · UC24 · UC25	
--	--

Endpoints REST	POST /missions/demarrer · GET /missions/{id} PUT /missions/{id}/terminer · GET /missions?presta_id=&client_id= GET /missions/{id}/suivi · SSE /missions/{id}/stream (temps réel) POST /missions/{id}/etape · GET /missions/{id}/etapes
RabbitMQ PRODUIT	Routing keys: mission.started, mission.completed, mission.etape.added
RabbitMQ CONSOMME	Routing key: devis.validated → crée automatiquement la mission (consumer de la queue q.mission.from.devis)
Tables BDD	missions(id, devis_id, client_id, presta_id, statut, date_debut, date_fin_prevue, date_fin_reelle, adresse) etapes_mission(id, mission_id, description, statut, timestamp)
Responsable	Membre 3

:8085 notation-service	UC: UC11 · UC26
Endpoints REST	POST /notations · GET /notations?cible_id= GET /notations/moyenne/{user_id} · GET /notations/mission/{mission_id} GET /notations/recues/{presta_id} · GET /notations/donnees/{client_id}
RabbitMQ PRODUIT	Routing key: notation.created
RabbitMQ CONSOMME	Routing key: mission.completed → autorise la notation (fenêtre de 48h)
Tables BDD	notations(id, mission_id, auteur_id, cible_id, role_auteur, note INT 1-5, commentaire, created_at, mission_completed_at)
Responsable	Membre 3

:8086 litige-service	UC: UC12 · UC34 · UC36-37-38-39
Endpoints REST	POST /litiges · GET /litiges/{id} · GET /litiges?statut=&mission_id= PUT /litiges/{id}/prendre-en-charge (agent SC) PUT /litiges/{id}/resoudre · PUT /litiges/{id}/escalader POST /litiges/{id}/message · GET /litiges/{id}/messages GET /litiges/stats (admin) · PUT /litiges/{id}/cloturer (admin)
RabbitMQ PRODUIT	Routing keys: litige.created, litige.resolved, litige.escalated
RabbitMQ CONSOMME	Aucun
Tables BDD	litiges(id, mission_id, auteur_id, type_litige, description, statut, agent_id, resolution, created_at, resolved_at) litige_messages(id, litige_id, auteur_id, contenu, created_at)
Responsable	Membre 4

:8087 urgence-service	UC: UC13-14-15 · UC16
Endpoints REST	POST /urgences · GET /urgences/prestataires?zone=&metier;=&radius;_km= GET /urgences/{id} · PUT /urgences/{id}/contacter PUT /urgences/{id}/accepter · PUT /urgences/{id}/refuser GET /urgences/{id}/statut
RabbitMQ PRODUIT	Routing key: urgence.created, urgence.accepted
RabbitMQ CONSOMME	Aucun
Tables BDD	urgences(id, client_id, metier, description, zone, latitude, longitude, statut, presta_acceptant_id, created_at) urgence_contacts(id, urgence_id, presta_id, statut_reponse, contacted_at)
Responsable	Membre 4

:8088 paiement-service		UC: UC27 · UC32 · UC40-41-42
Endpoints REST	POST /paiements/initier · GET /paiements/{id} POST /paiements/{id}/confirmer · GET /paiements/mission/{mission_id} GET /paiements/presta/{presta_id} (historique gains UC27) GET /commissions · PUT /commissions/taux (admin UC32) GET /paiements/monitoring (admin UC40-42) POST /paiements/webhook (callback MTN/Orange)	
RabbitMQ PRODUIT	Routing key: paiement.initiated, paiement.confirmed, paiement.failed	
RabbitMQ CONSOMME	Routing key: mission.completed → déclenche l'initialisation du paiement	
Tables BDD	paiements(id, mission_id, client_id, presta_id, montant_total, commission_taux, montant_presta, operateur, tel_payeur, ref_transaction, statut, initiated_at, confirmed_at) commissions(id, taux_pct, effectif_depuis, cree_par)	
Responsable	Membre 4 (semaines 2-3)	

:8089 chat-service		UC: UC5
Endpoints REST	WebSocket STOMP /ws (connexion) SUBSCRIBE /topic/chat/{mission_id} (recevoir messages) SEND /app/chat/{mission_id} (envoyer message) GET /chat/{mission_id}/historique (messages REST) GET /chat/{mission_id}/messages?page=0	
RabbitMQ PRODUIT	Aucun (WebSocket direct)	
RabbitMQ CONSOMME	Aucun	
Tables BDD	chat_messages(id, mission_id, expéditeur_id, rôle_expéditeur, contenu, type_message, lu, created_at)	
Responsable	Membre 1 (semaine 3 — après auth + gateway stabilisés)	

:8090 notification-service		UC: Transversal — tous UC
Endpoints REST	GET /notifications/{user_id} (historique) PUT /notifications/{id}/lire · GET /notifications/{user_id}/non-lues POST /notifications/test (admin — test SMS) Aucun endpoint public métier — service interne	
RabbitMQ PRODUIT	Aucun (service consommateur uniquement)	
RabbitMQ CONSOMME	Routing keys consommés: user.registered, prestataire.validated, demande.created, devis.created, devis.validated, mission.started, mission.completed, litige.created, litige.resolved, urgence.created, paiement.confirmed	
Tables BDD	notifications(id, user_id, titre, contenu, type_notif, canal (SMS/PUSH/IN_APP), lu, sent_at, created_at)	
Responsable	Membre 3	

4. Architecture RabbitMQ — Configuration complète

RabbitMQ remplace Kafka avec une logique d'échanges et de queues. L'échange principal est de type 'topic', ce qui permet un routage flexible par routing key avec wildcards (*, #). Contrairement à Kafka, les messages sont consommés et supprimés — pas de log persistant. C'est plus simple pour des novices.

4.1 Configuration de l'échange principal

```
Exchange : serviloc.events
Type : topic (routage par pattern de routing key)
Durable : true (survit aux redémarrages RabbitMQ)
Auto-del : false

Routing key convention : <domaine>.<action>
Exemples : user.registered · demande.created · mission.completed
Wildcards : user.* correspond à user.registered, user.updated, etc.
# correspond à tout ce qui suit
```

4.2 Queues et bindings complets

Queue	Binding (routing key)	Consommateur	Rôle
q.user.profile.create	user.registered	user-service	Créer profil vide après inscription
q.mission.from.devis	devis.validated	mission-service	Créer automatiquement la mission
q.paiement.from.mission	mission.completed	paiement-service	Initier le paiement à la fin mission
q.notif.user	user.*	notification-service	SMS/push sur événements utilisateur
q.notif.demande	demande.#	notification-service	Alertes demandes et devis
q.notif.mission	mission.*	notification-service	Alertes cycle de vie mission
q.notif.litige	litige.*	notification-service	Alertes litiges et résolutions
q.notif.urgence	urgence.*	notification-service	Alertes mode urgence
q.notif.paiement	paiement.*	notification-service	Confirmation et échec paiement
q.notif.prestataire	prestataire.*	notification-service	Validation/rejet dossier prestataire

4.3 Patron de code RabbitMQ à reproduire

Ce patron est la seule façon d'utiliser RabbitMQ dans ce projet. Toute variation doit être validée par le Membre 1.

```

// ■■ CONFIGURATION (RabbitMQConfig.java – dans chaque service) ■■
@Configuration
public class RabbitMQConfig {
    public static final String EXCHANGE = "serviloc.events";

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(EXCHANGE, true, false);
    }

    // Dans notification-service uniquement – déclarer toutes les queues
    @Bean public Queue qNotifUser() { return new Queue("q.notif.user", true); }
    @Bean public Queue qNotifDemande() { return new Queue("q.notif.demande", true); }
    // ... autres queues

    @Bean
    public Binding bindingUser(Queue qNotifUser, TopicExchange exchange) {
        return BindingBuilder.bind(qNotifUser).to(exchange).with("user.*");
    }
}

// ■■ PRODUCER (dans demande-service par exemple) ■■
@Service
public class DemandeService {
    @Autowired RabbitTemplate rabbitTemplate;

    public Demande creerDemande(DemandeRequest req) {
        Demande d = demandeRepo.save(mapper.toEntity(req));
        DemandeEvent evt = new DemandeEvent(d.getId(), d.getClientId(), d.getMetier());
        rabbitTemplate.convertAndSend(
            RabbitMQConfig.EXCHANGE, "demande.created", evt);
        return d;
    }
}

// ■■ CONSUMER (dans notification-service) ■■
@Component
public class DemandeListener {
    @RabbitListener(queues = "q.notif.demande")
    public void onDemandeEvent(DemandeEvent event) {
        // Envoyer SMS aux prestataires disponibles dans la zone
        smsService.send(event.getZone(), "Nouvelle demande : " + event.getMetier());
    }
}

```


5. Répartition de l'équipe — 4 membres

Avec 10 microservices pour 4 membres, le regroupement est obligatoire. Chaque membre est PROPRIÉTAIRE de ses services — personne d'autre n'y touche sans accord explicite.

Membre	Services en charge	UC couverts	Charge relative
Membre 1 (Lead)	auth-service api-gateway eureka-server Docker Compose infra chat-service (S3)	UC1, UC5 UC29, UC35 + infra	■ La plus lourde Bloquant pour tous
Membre 2	user-service demande-service	UC2-4, UC6-8 UC17-18, UC19-23 UC28*, UC30, UC31	■ Élevée 9 UC à couvrir
Membre 3	mission-service notation-service notification-service	UC9-11, UC24-26 + notifications transversales	■ Élevée Consommateur RMQ
Membre 4	litige-service urgence-service paiement-service	UC12-16 UC27, UC32 UC34, UC36-42	■ Élevée 3 domaines métier

6. Plan d'exécution — 4 semaines détaillées

Ce plan suit une logique de dépendances strictes : on ne peut pas coder les services métier sans l'auth, on ne peut pas tester RabbitMQ sans la configuration de l'échange. L'ordre est non négociable.

SEMAINE 1 Fondations — rien d'autre n'est possible sans ça

■ Jours 1-2 — Réunion de lancement complète (tous membres + équipe frontend)

Produire API_CONTRACT.md v1 avec : tous les endpoints, format exact des JSON request/response, codes d'erreur standardisés (ex: 404 USER_NOT_FOUND, 403 ACCESS_DENIED), système d'authentification (Bearer JWT dans Authorization header). Ce document est partagé avec l'équipe frontend. Toute modification nécessite notification 48h à l'avance.

■ Jours 1-2 — Environnement de travail (Membre 1 + tous)

Installer Java 21 LTS, Maven 3.9+, IntelliJ IDEA Community, Docker Desktop, Postman sur toutes les machines. Créer le repo GitHub avec structure multi-modules Maven : serviloc-backend/ contenant un sous-module par service. Configurer les branches : main (protégée), develop, feature/[nom-service].

■ Jours 1-3 — Docker Compose infrastructure (Membre 1)

Fichier docker-compose.yml contenant : RabbitMQ :5672 + Management UI :15672, 9 instances PostgreSQL (une par service, ports 5433 à 5441), Eureka Server, Redis :6379 (cache sessions optionnel). 'docker-compose up -d' doit fonctionner sur TOUTES les machines avant fin jour 3. Tester sur Windows ET Mac/Linux — les volumes Docker se comportent différemment.

■ Jours 1-3 — RabbitMQ setup (Membre 1)

Créer l'échange 'serviloc.events' (type: topic, durable: true) via le Management UI. Créer toutes les queues et tous les bindings définis en section 4.2. Exporter la configuration RabbitMQ en JSON et la committer dans le repo (dossier /infra/rabbitmq/definitions.json). Cela permet à n'importe quel membre de recréer l'infra en 5 minutes.

■ Jours 3-5 — auth-service COMPLET (Membre 1 — priorité absolue)

POST /auth/register : valider email/tel, hasher password (BCrypt), créer user en BDD, générer OTP 6 chiffres, envoyer SMS via Africa's Talking sandbox, publier user.registered. POST /auth/verify-otp : valider OTP (expiry 10 min), marquer is_verified=true. POST /auth/login : vérifier credentials + is_verified, retourner access_token (15 min) + refresh_token (7 jours). POST /auth/refresh-token : renouveler access_token. LIVRABLE : curl -X POST /auth/login retourne un JWT valide avant vendredi 17h.

■ Jours 3-5 — Premiers services (Membres 2, 3, 4)

Pendant que Membre 1 code l'auth, les autres créent leur module Spring Boot de base : connexion à leur BDD PostgreSQL dédiée, une entité JPA simple, un repository, un controller avec GET /health qui retourne {status: UP}. Pas de logique métier cette semaine — juste faire tourner le service et le voir s'enregistrer sur Eureka.

■ Vendredi S1 — Livrable de synchronisation

auth-service : POST /auth/login retourne JWT ✓. docker-compose up fonctionne sur toutes les machines ✓. Eureka UI montre tous les services enregistrés ✓. API_CONTRACT.md v1 partagé avec frontend ✓. Si l'un de ces livrables manque, le weekend est utilisé pour le corriger.

SEMAINE 2 Services core P1 — les fonctionnalités essentielles

■ Membre 1 — API Gateway + filtre JWT

Configurer Spring Cloud Gateway avec les règles de routage (section API Gateway du doc précédent). Implémenter le GlobalFilter JWT : extraire le Bearer token, valider la signature, injecter les claims (userId, role) dans les headers X-User-Id et X-User-Role vers les services downstream. Tester que /users/** est protégé et que /auth/** est public.

■ Membre 2 — user-service complet

Tous les endpoints profils client et prestataire. Consumer RabbitMQ q.user.profile.create → créer le profil vide après inscription. Upload documents prestataire (stocker en base64 ou URL externe). Endpoints admin : GET /users (liste paginée), PUT /prestataires/{id}/valider, publier prestataire.validated sur RabbitMQ.

■ Membre 2 — demande-service : création et liste

POST /demandes avec validation (zone, métier, description obligatoires). GET /demandes filtré par statut/zone/métier pour les prestataires (UC19-21). GET /demandes pour le client (ses propres demandes). Publier demande.created sur RabbitMQ à chaque création.

■ Membre 3 — mission-service : consumer RabbitMQ + démarrage

Consumer queue q.mission.from.devis (routing key: devis.validated) : créer automatiquement la mission avec statut PLANIFIEE. POST /missions/demarrer : passer statut à EN_COURS, enregistrer date_debut. GET /missions/{id} avec détail complet. Publier mission.started sur RabbitMQ.

■ Membre 3 — notification-service : premiers consumers

Consumer q.notif.user (user.registered) → SMS de bienvenue via Africa's Talking. Consumer q.notif.demande (demande.created) → SMS aux prestataires de la zone. Consumer q.notif.demande (devis.validated) → SMS au client 'Votre devis a été accepté'. Enregistrer chaque notification envoyée en BDD.

■ Membre 4 — litige-service complet

POST /litiges (client signale un litige sur une mission). GET /litiges avec filtres (statut, mission_id). PUT /litiges/{id}/prendre-en-charge (agent service client). PUT /litiges/{id}/resoudre avec message de résolution. POST/GET /litiges/{id}/messages (fil de discussion). Publier litige.created et litige.resolved.

■ Point de sync vendredi S2 (frontend + backend — 1h)

Le frontend doit pouvoir tester en live : inscription → OTP → connexion → voir son profil → créer une demande → voir les devis disponibles. Identifier tous les désalignements de format JSON et les corriger avant S3.

SEMAINE 3 Services P2 + intégration end-to-end

■ Membre 1 — chat-service (WebSocket STOMP)

Ajouter spring-boot-starter-websocket. Configurer le WebSocket endpoint /ws avec STOMP. Implémenter : SEND /app/chat/{mission_id} pour envoyer, SUBSCRIBE /topic/chat/{mission_id} pour recevoir en temps réel. GET /chat/{mission_id}/historique pour charger les anciens messages. NOTE : Le chat ne fonctionne QUE sur une mission active (mission_id requis).

■ Membre 2 — demande-service : devis complet

POST /demandes/{id}/devis (prestataire crée un devis) → publier devis.created. GET /demandes/{id}/devis (liste des devis pour cette demande). PUT /devis/{id}/valider (client accepte) → publier devis.validated → déclenche création mission via RabbitMQ. PUT /devis/{id}/refuser (client refuse) → publier devis.refused.

■ Membre 3 — mission-service : suivi + fin

PUT /missions/{id}/terminer : passer statut TERMINEE, enregistrer date_fin_reelle, publier mission.completed → déclenche paiement et ouverture notation. POST /missions/{id}/etape : ajouter une étape de progression. SSE /missions/{id}/stream : Server-Sent Events pour le suivi temps réel (UC9-10). Utiliser SseEmitter de Spring.

■ Membre 3 — notation-service

POST /notations : vérifier que la mission est TERMINEE et que l'auteur est bien client ou prestataire de cette mission. Vérifier fenêtre de 48h. GET /notations/moyenne/{user_id} : calculer et retourner la note moyenne. Appel Feign vers user-service pour mettre à jour note_moyenne sur le profil prestataire. Publier notation.created.

■ Membre 4 — urgence-service

POST /urgences : créer demande urgente avec géolocalisation (lat/lng). GET /urgences/prestataires : requête géographique sur la table prestataires (distance via formule Haversine en SQL ou Feign vers user-service). PUT /urgences/{id}/contacter : notifier le prestataire. PUT /urgences/{id}/accepter / /refuser. Publier urgence.created et urgence.accepted.

■ Membre 4 — paiement-service

Consumer queue q.paiement.from.mission (mission.completed) → créer l'entrée paiement. POST /paiements/initier : appel API Africa's Talking Mobile Money sandbox. POST /paiements/webhook : recevoir le callback de confirmation (MTN/Orange). GET /paiements/presta/{presta_id} : historique gains (UC27). GET/PUT /commissions : gestion taux de commission (UC32). GET /paiements/monitoring : dashboard admin (UC40-42).

■ Tous — tests d'intégration cross-services

Tester le flow complet avec les deux équipes : inscription → demande → devis → validation → mission → suivi → fin → notation → paiement. Chaque étape doit déclencher les bons événements RabbitMQ et les bonnes notifications SMS. Utiliser Postman Collections partagées pour reproduire les tests.

■ Tous — Swagger OpenAPI

Ajouter springdoc-openapi-starter-webmvc-ui dans tous les services. Chaque service expose /swagger-ui.html et /v3/api-docs. Annoter les controllers avec @Operation et @ApiResponse. L'URL Swagger de chaque service est documentée dans le README.

SEMAINE 4 Stabilisation · Déploiement · Démo

■ Jours 1-2 — Corrections uniquement

Aucune nouvelle feature. Traiter les bugs en priorité décroissante : P1 = bloque la démo, P2 = fonctionnalité dégradée, P3 = cosmétique. Seuls les bugs P1 sont obligatoires à corriger. Utiliser un board simple (GitHub Issues ou Notion) pour tracker les bugs.

■ Jour 2 — Déploiement progressif sur Railway

Ordre STRICT de déploiement : PostgreSQL (toutes instances) → RabbitMQ → Eureka Server → auth-service → tester auth en prod → user-service → demande-service → mission-service → notation-service → litige-service → paiement-service → urgence-service → notification-service → chat-service → api-gateway. Ne passer au service suivant QUE si le précédent répond correctement.

■ Jour 3 — Variables d'environnement production

Configurer dans Railway pour chaque service : DATABASE_URL, RABBITMQ_URL, EUREKA_SERVER_URL, JWT_SECRET, AFRICAS_TALKING_API_KEY, AFRICAS_TALKING_USERNAME, SPRING_PROFILES_ACTIVE=prod. Vérifier que aucune valeur sensible n'est dans le code source.

■ Jour 3 — Tests end-to-end en production

Reproduire le scénario de démo complet sur l'environnement Railway (pas localhost). Vérifier : CORS configuré pour l'URL du frontend en production, JWT valide en prod, RabbitMQ accessible depuis tous les services, SMS réellement envoyés via Africa's Talking.

■ Jour 4 — Scénario de démo préparé

Définir et documenter le flow exact de la démo avec données réalistes : noms camerounais, zones (Yaoundé/Douala), métiers (électricien, plombier...), montants en XAF. Répéter le scénario 2 fois avec l'équipe frontend. Préparer un compte de chaque rôle pré-crée (client, prestataire, admin, SC).

■ Jour 5 — Buffer et documentation

Réservé aux imprévus critiques. Si tout est stable : compléter le README (prérequis, installation locale, variables d'environnement, démarrage docker-compose, ordre de lancement des services). Ce README doit permettre à quelqu'un d'externe de lancer le projet en 30 minutes.

7. Schéma de base de données — Vue consolidée

RÈGLE FONDAMENTALE : jamais de clé étrangère entre deux bases différentes. Les IDs sont stockés comme simples colonnes bigint — la cohérence est garantie par la logique applicative et les événements RabbitMQ.

BDD	Tables	Relations clés
serviloc_auth (auth-service)	users	id → référencé partout (user_id logique) role: CLIENT PRESTA ADMIN SC
serviloc_user (user-service)	user_profiles prestataire_profiles documents	user_id = users.id (logique, pas FK) prestataire_profiles.user_id → user_profiles.user_id
serviloc_demande (demande-service)	demandes devis	demandes.client_id → users.id (logique) devis.demande_id → demandes.id (réelle) devis.presta_id → users.id (logique)
serviloc_mission (mission-service)	missions etapes_mission	missions.devis_id → devis.id (logique) missions.client_id, presta_id (logiques) etapes_mission.mission_id → missions.id (réelle)
serviloc_notation (notation-service)	notations	notations.mission_id (logique) auteur_id, cible_id → users.id (logiques) UNIQUE(mission_id, auteur_id)
serviloc_litige (litige-service)	litiges litige_messages	litiges.mission_id (logique) litige_messages.litige_id → litiges.id (réelle)
serviloc_urgence (urgence-service)	urgences urgence_contacts	urgences.client_id (logique) urgence_contacts.urgence_id → urgences.id (réelle)
serviloc_paiement (paiement-service)	paiements commissions	paiements.mission_id, client_id, presta_id (logiques) commissions: historique des taux
serviloc_chat (chat-service)	chat_messages	chat_messages.mission_id (logique) expéditeur_id → users.id (logique)
serviloc_notif (notification-service)	notifications	notifications.user_id (logique) canal: SMS PUSH IN_APP

8. Dépendances Maven par service

Dépendances communes à TOUS les services

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- postgresql (scope: runtime)
- lombok
- mapstruct (org.mapstruct)
- springdoc-openapi-starter-webmvc-ui (3.x)
- spring-cloud-starter-netflix-eureka-client
- spring-amqp + spring-rabbit (org.springframework.amqp)

auth-service uniquement

- spring-boot-starter-security
- jjwt-api + jjwt-impl + jjwt-jackson (io.jsonwebtoken – version 0.12.x)
- african-talking (com.africastalking – pour OTP SMS)

Services qui font des appels inter-services (user, demande, mission, notation, paiement, urgence)

- spring-cloud-starter-openfeign

mission-service uniquement (Server-Sent Events, pas de dépendance extra)

- SseEmitter est inclus dans spring-boot-starter-web – aucune dépendance supplémentaire.

chat-service uniquement

- spring-boot-starter-websocket
- NB: spring-boot-starter-web DOIT AUSSI être présent (WebSocket + REST coexistent)

paiement-service uniquement

- african-talking (SDK Mobile Money)
- spring-boot-starter-web (webhook endpoint pour callback opérateurs)

api-gateway — ATTENTION règles spéciales

- spring-cloud-starter-gateway (WebFlux – réactif)
- spring-cloud-starter-netflix-eureka-client
- jjwt-api + jjwt-impl (pour valider JWT dans le GlobalFilter)
- INTERDICTION : ne JAMAIS ajouter spring-boot-starter-web (incompatible WebFlux)

9. Carte des risques — Backend complet

Niv.	Risque	Impact	Mitigation
■ FATAL	auth-service non livré fin S1	Tout le reste est bloqué — personne ne peut tester la sécurité de son service.	Membre 1 n'a QUE auth-service en S1. Aucune autre tâche.
■ FATAL	RabbitMQ mal configuré dès le départ	Exchange ou bindings incorrects → les consumers ne reçoivent rien → notification-service inopérant, mission non créée automatiquement.	Membre 1 exporte la config RabbitMQ en JSON committée dans le repo. Vérifier chaque queue avec le Management UI avant S2.
■ CRITIQUE	10 services pour 4 novices = abandon P3 quasi certain	Chat WebSocket et urgences géolocalisées risquent de ne jamais être finis.	Priorisation stricte P1→P2→P3. Démo possible sans chat et sans urgences si P1+P2 sont complets.
■ ÉLEVÉ	Appels Feign entre services non testés	mission-service appelle user-service via Feign pour mettre à jour la note — si un service est down, le Feign échoue et l'endpoint entier plante.	Ajouter @FeignClient avec fallback pour chaque appel inter-service. Tester les scénarios de service indisponible dès S2.
■ ÉLEVÉ	Paiements Mobile Money non fonctionnels en démo	Les APIs MTN MoMo et Orange Money demandent un compte marchand avec validation KYC — délai de plusieurs semaines.	Utiliser Africa's Talking sandbox uniquement. Le paiement en démo = simulation avec statut CONFIRME automatique après 5 secondes.
■ ÉLEVÉ	WebSocket bloqué par la Gateway	Spring Cloud Gateway nécessite une configuration spéciale pour proxifier les connexions WebSocket (upgrade HTTP → WS).	Ajouter dans la Gateway : filters: - PreserveHostHeader. Tester la connexion WebSocket via la Gateway dès que chat-service est créé.
■ ÉLEVÉ	Désalignement format JSON avec le frontend	Un champ renommé côté backend sans notifier le frontend = 1-2 jours de débogage pour retrouver l'origine.	API_CONTRACT.md versionné. Toute modification = PR + notification Slack/WhatsApp. Point de sync frontend/backend chaque vendredi.
■ MODÉRÉ	SSE (Server-Sent Events) pour le suivi mission	SseEmitter peut timeout ou être coupé par des proxies. En prod sur Railway, les connexions longues sont parfois coupées à 30s.	Implémenter un heartbeat SSE toutes les 15 secondes. Le frontend doit gérer la reconnexion automatique.
■ MODÉRÉ	Géolocalisation urgences sans PostGIS	La formule Haversine en SQL pur est lente sur beaucoup de données. PostGIS n'est pas disponible sur Railway gratuit.	Implémentation Haversine en Java côté service — acceptable pour une démo avec un nombre limité de prestataires.

10. Règles de fonctionnement de l'équipe

Ces règles ne sont pas des suggestions. Leur non-respect dans les premiers projets microservices est la cause principale d'échec.

■ Variables d'environnement dès le Jour 1

JAMAIS de credentials en dur dans le code. Fichier `.env` local + `.gitignore`. `application-dev.yml` pour le dev, `application-prod.yml` pour Railway. `JWT_SECRET` minimum 64 caractères alphanumériques.

■ API Contract = document vivant

Toute modification d'un endpoint existant nécessite : mise à jour `API_CONTRACT.md`, PR sur GitHub, notification WhatsApp à toute l'équipe frontend et backend, délai minimum 24h avant déploiement.

■ Git : 1 branche par service

Convention : `feature/auth-service`, `feature/user-service`... Commits atomiques avec message clair : `'feat(auth): add refresh token endpoint'`. PR obligatoire pour merger dans `develop`. Le Membre 1 valide toutes les PR.

■■ Daily standup 15 min — maximum

Format unique : 'J'ai fini X. Je travaille sur Y. Je suis bloqué sur Z.' Si quelqu'un est bloqué depuis plus de 2h : demander de l'aide immédiatement. Pas de réunion > 15 min sans ordre du jour écrit.

■ Tester dans Postman avant de dire que c'est fini

Un endpoint est considéré livré seulement quand : la requête Postman passe, le cas d'erreur renvoie le bon code HTTP, et la Collection Postman est mise à jour.

■■ Ne jamais partir en prod le vendredi

Tous les déploiements Railway se font en semaine 4, du lundi au jeudi. Le vendredi est réservé aux tests et corrections post-déploiement.

Verdict final — Ce plan est-il réaliste ?

Honnêtement : c'est le maximum de ce qui est livrable avec 4 novices en 1 mois. Les services P1 (auth, user, demande, mission, gateway) forment une application fonctionnelle et démontrable. Les services P2 (notation, litige, paiement, notification, urgences) enrichissent le produit si P1 est livré à temps. Le chat (P3) est un bonus. Si vous essayez de tout livrer en même temps sans priorisation, vous obtiendrez 10 services à 30% — rien de démontrable. La discipline de priorisation est plus critique que la compétence technique.